Claude Code Workshop — Implementation Plan

> **For agentic workers:** REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (`- [ ]`) syntax for tracking.

Goal: Build a complete, hands-on, 6-hour Claude Code workshop for non-technical professionals — a Quarto/reveal.js deck, a pre-executed token-consumption notebook, six copy-paste exercise cards, deterministic fallbacks, and instructor notes — all under a new `claude-code-workshop/` directory.

Architecture: A single business deliverable (monthly sales report on Adriatica `orders.csv`) grows across six blocks; each Claude Code capability (loop, goal, efforts, token cost, MCP, GitHub, `/loop`) and Google Antigravity is introduced where the project needs it. Build artifacts are emitted/checked by scripts under `claude-code-workshop/_build/` (mirroring the repo's `workshops/_build/` convention); each task ends with a runnable verification.

Tech Stack: Quarto (reveal.js), Python 3.13 (pandas, matplotlib, nbformat, anthropic), Markdown. Claude Code (Terminal CLI) and Google Antigravity are the subject tools, not build dependencies.

Spec: `docs/superpowers/specs/2026-06-19-claude-code-workshop-design.md`

Global Constraints

- **Language:** English for all participant-facing materials.
- **Location:** everything under `claude-code-workshop/` at the repo root. Do NOT place under `workshops/lab-aNN/` — this is a standalone professional workshop, outside the IMB2026 free-tools student track.
- **Accuracy (no memory facts):** model IDs, per-token pricing, token/usage field names, Claude Code command syntax (efforts, `/fast`, `/loop`, `claude mcp add`, `/cost`, `/context`, `/compact`), and Antigravity free-tier limits MUST be verified at build time via the `claude-api` skill, the running `claude` CLI, and `https://docs.claude.com/`. Re-verify Antigravity limits ~1 week before delivery (per `planning/resources.md`; quota moved to weekly, no committed permanent free tier).
- **Timing is instructor discretion:** suggested block durations live ONLY in `instructor-notes.md` as an adjustable template. Do NOT hard-code clock times or fixed minute counts into the deck or cards.
- **Data:** reuse Adriatica `orders.csv` (copied into `data/`, self-contained). Columns: `order_id, order_date, customer_id, product_id, product_name, category, quantity, unit_price`. Revenue = `quantity * unit_price`. Raw file = 4644 rows. Seasonal peak month = `2024-12`.
- **Deterministic fallbacks:** every live-AI step has a pre-written known-good artifact under `fallbacks/`.
- **Notebook is read-don't-run:** committed pre-executed; its presentation layer (table + charts) must run WITHOUT an API key from a committed CSV.
- **Git:** branch off `main` before committing (never commit straight to `main`); commit after each task. Obtain the user's go-ahead before the first commit.
- **Card schema (verbatim, used by the linter):** every `exercises/b*.md` must contain these four H2 headings: `## Goal`, `## Type this`, `## You should see`, `## If it looks wrong`.

Task 1: Scaffold bundle + copy data + data check

Files:

- Create: `claude-code-workshop/` with subdirs `slides/ notebook/ exercises/ fallbacks/ data/ _build/`
- Create: `claude-code-workshop/data/orders.csv` (copy of `workshops/lab-a01-pandas-business-data/data/orders.csv`)
- Create/Test: `claude-code-workshop/_build/check_data.py`

Interfaces:

- Produces: `data/orders.csv` (canonical workshop dataset); `_build/check_data.py` (importable `monthly_revenue(patl -> pandas.Series` and a `__main__` self-check).

- [ ] **Step 1: Create directories and copy data**

```
mkdir -p claude-code-workshop/{slides,notebook,exercises,fallbacks,data,_build}
cp workshops/lab-a01-pandas-business-data/data/orders.csv claude-code-workshop/data/orders.csv
```

☐ **Step 2: Write the data check (failing — script does not exist yet)**

Create `claude-code-workshop/_build/check_data.py`:

```
"""Verify the workshop's copy of orders.csv and the seasonal signal the
workshop relies on. Run: python claude-code-workshop/_build/check_data.py"""
from pathlib import Path
import pandas as pd

DATA = Path(__file__).resolve().parents[1] / "data" / "orders.csv"
EXPECTED_COLS = ["order_id", "order_date", "customer_id", "product_id",
                 "product_name", "category", "quantity", "unit_price"]

def monthly_revenue(path: Path) -> pd.Series:
    df = pd.read_csv(path)
    df["revenue"] = df["quantity"] * df["unit_price"]
    df["month"] = df["order_date"].str.slice(0, 7)
    return df.groupby("month")["revenue"].sum().sort_index()

def main() -> None:
    df = pd.read_csv(DATA)
    assert list(df.columns) == EXPECTED_COLS, f"columns drifted: {list(df.columns)}"
    assert len(df) == 4644, f"expected 4644 rows, got {len(df)}"
    mr = monthly_revenue(DATA)
    best = mr.idxmax()
    assert best == "2024-12", f"expected peak 2024-12, got {best}"
    assert mr["2024-12"] > mr["2024-07"], "December should beat July (seasonal signal)"
    print(f"DATA OK  rows={len(df)}  peak={best}  peak_rev={mr[best]:.2f}")

if __name__ == "__main__":
    main()
```

☐ **Step 3: Run it**

Run: `python claude-code-workshop/_build/check_data.py` Expected: `DATA OK  rows=4644  peak=2024-12 peak_rev=...`

☐ **Step 4: Commit**

```
git add claude-code-workshop/data/orders.csv claude-code-workshop/_build/check_data.py
git commit -m "feat(cc-workshop): scaffold bundle, copy data, add data check"
```

Task 2: Card linter + Block 1 card

Files:

- Create/Test: `claude-code-workshop/_build/check_cards.py`
- Create: `claude-code-workshop/exercises/b1-setup.md`

Interfaces:

- Produces: `_build/check_cards.py` (`__main__` lints every existing `exercises/b*.md` against the four-heading card schema; exits non-zero on any violation).

☐ **Step 1: Write the card linter (failing — no cards yet)**

Create `claude-code-workshop/_build/check_cards.py`:

```python
"""Lint exercise cards against the workshop card schema.
Run: python claude-code-workshop/_build/check_cards.py"""
from pathlib import Path
import sys

CARDS = sorted((Path(__file__).resolve().parents[1] / "exercises").glob("b*.md"))
REQUIRED = ["## Goal", "## Type this", "## You should see", "## If it looks wrong"]

def main() -> int:
    if not CARDS:
        print("FAIL: no exercise cards found")
        return 1
    bad = []
    for card in CARDS:
        text = card.read_text(encoding="utf-8")
        missing = [h for h in REQUIRED if h not in text]
        if missing:
            bad.append((card.name, missing))
    for name, missing in bad:
        print(f"FAIL {name}: missing {missing}")
    if bad:
        return 1
    print(f"CARDS OK  ({len(CARDS)} cards)")
    return 0

if __name__ == "__main__":
    sys.exit(main())
```

☐ **Step 2: Run it to confirm it fails**

Run: `python claude-code-workshop/_build/check_cards.py` Expected: `FAIL: no exercise cards found` (exit 1)

☐ **Step 3: Write the Block 1 card**

Create `claude-code-workshop/exercises/b1-setup.md`. Content (full card — this is the model all other cards follow):

```markdown
# Block 1 — Doorway & Setup

## Goal
Get Claude Code running in your terminal and have your first conversation. By the
end you will have typed `claude` and asked it to describe a folder — proving the
terminal is just a place to *talk to an assistant*, not to memorise commands.

## Type this
1. Open your terminal (facilitator will show where).
2. Confirm the tool is installed:
   `claude --version`
3. Start it:
   `claude`
4. In the chat, type:
   `What files are in this folder, and what do they look like they're for?`

## You should see
- A version number on `claude --version`.
- After `claude`, a welcome prompt where you can type in plain English.
- A short, plain-language summary of the folder's contents.
```

```
## If it looks wrong
- `command not found` → tell the facilitator; do the install step together.
- It asks you to log in → follow the browser sign-in, then return to the terminal.
- It does nothing → say in the chat: `List the files here and explain them simply.`
```

☐ **Step 4: Run the linter to verify it passes for b1**

Run: `python claude-code-workshop/_build/check_cards.py` Expected: `CARDS OK  (1 cards)`

☐ **Step 5: Commit**

```
git add claude-code-workshop/_build/check_cards.py claude-code-workshop/exercises/b1-setup.md
git commit -m "feat(cc-workshop): add card linter and Block 1 setup card"
```

Task 3: Exercise cards b2–b6

Files:

- Create: `claude-code-workshop/exercises/b2-loop-and-goal.md`
- Create: `claude-code-workshop/exercises/b3-efforts-and-tokens.md`
- Create: `claude-code-workshop/exercises/b4-mcp-and-github.md`
- Create: `claude-code-workshop/exercises/b5-antigravity.md`
- Create: `claude-code-workshop/exercises/b6-loop-automation.md`

Interfaces:

- Consumes: card schema enforced by `_build/check_cards.py` (Task 2).
- Produces: the five remaining cards. Each MUST contain the four required H2 headings and follow the b1 voice (plain English, exact text to type, recovery line).

Author each card with the exact copy-paste prompts below (these are the canonical prompts; the deck's "□ Your turn" slides reuse them).

☐ **Step 1: Write `b2-loop-and-goal.md`**

Goal: set a goal and watch the agentic loop. Key "Type this" prompts:

- `Here is a sales file: data/orders.csv. I want a monthly sales report — total revenue per month, and which month was best. Plan your approach first, then do it.`
- (after the plan) `Yes, go ahead.`
- Verification prompt: `What was the best month, and what was its revenue?` → expect **2024-12**. "You should see": a plan, then Claude writing and running Python, then a monthly table peaking at 2024-12. "If it looks wrong": `That doesn't match — December should be the best month. Re-check how you computed revenue (quantity times unit_price).`

☐ **Step 2: Write `b3-efforts-and-tokens.md`**

Goal: feel effort/model trade-offs and read cost. Key "Type this":

- `Run the same monthly-report task again, but think harder about edge cases like duplicate orders.` (high effort framing)
- In-CLI commands to try: `/usage`, `/context`, `/compact` (use `/usage` — the built-in cost command; `/cost` is a plugin skill).
- Switch model/effort using the current Claude Code mechanism (verify exact UI at build — see Global Constraints). Note `/fast` for Opus. "You should see": different speed/verbosity; `/cost` showing tokens & spend climbing with effort. "If it looks wrong": `/cost` empty → run a task first, then `/cost`.

☐ **Step 3: Write `b4-mcp-and-github.md`**

Goal: add an MCP server and have Claude open a PR. Key "Type this":

- `claude mcp add ...` (exact syntax verified at build; filesystem MCP for the intro).

- `Create a new GitHub repository called adriatica-sales-report and push our report script to it.`
- `Open a pull request that adds a short README describing the monthly report.` "You should see": Claude calling MCP/gh, a repo URL, a PR URL. "If it looks wrong": not authenticated → `Help me sign in to GitHub first, then try again.` Fallback pointer: `fallbacks/b4-pr-example.md`.

☐ **Step 4: Write `b5-antigravity.md`**

Goal: start a project in Antigravity with MCP. Key "Type this" / steps:

- Open Antigravity, open the project folder.
- Add the same MCP server in Antigravity's MCP settings (exact path verified at build; use `fallbacks/b5-antigravity-mcp-config.json`).
- Agent prompt: `Open data/orders.csv and add a bar chart of monthly revenue to the report.` "You should see": the agent working in a visual IDE; a chart added. "If it looks wrong": quota exhausted → switch to the demo recording / `fallbacks/` artifact (note Antigravity's weekly free limit).

☐ **Step 5: Write `b6-loop-automation.md`**

Goal: make the report recur. Key "Type this":

- `Set up the monthly report to run on a schedule, every Monday morning, and tell me how it will run.` (uses the current `/loop` / scheduled-work mechanism — verify exact command at build).
- Wrap reflection prompt: `Summarise what we built today and what I should always double-check before trusting your output.` "You should see": a scheduled/recurring task described; a verify-everything summary. "If it looks wrong": scheduling unavailable in their setup → explain the concept and show the command in the notes.

☐ **Step 6: Run the linter**

Run: `python claude-code-workshop/_build/check_cards.py` Expected: `CARDS OK  (6 cards)`

☐ **Step 7: Commit**

```
git add claude-code-workshop/exercises/
git commit -m "feat(cc-workshop): add exercise cards b2-b6"
```

Task 4: Fallback artifacts + fallback check

Files:

- Create: `claude-code-workshop/fallbacks/b2-sales-summary.py`
- Create: `claude-code-workshop/fallbacks/b2-expected-output.md`
- Create: `claude-code-workshop/fallbacks/b4-pr-example.md`
- Create: `claude-code-workshop/fallbacks/b4-mcp-config.json`
- Create: `claude-code-workshop/fallbacks/b5-antigravity-mcp-config.json`
- Create/Test: `claude-code-workshop/_build/check_fallbacks.py`

Interfaces:

- Consumes: `data/orders.csv` (Task 1), `monthly_revenue` logic.
- Produces: known-good artifacts; `check_fallbacks.py` runs `b2-sales-summary.py` and asserts the December peak.

☐ **Step 1: Write the known-good Block 2 script**

Create `claude-code-workshop/fallbacks/b2-sales-summary.py`:

```
"""Known-good monthly sales summary (fallback if live generation fails).
Run: python claude-code-workshop/fallbacks/b2-sales-summary.py"""
from pathlib import Path
import pandas as pd

DATA = Path(__file__).resolve().parents[1] / "data" / "orders.csv"
```

```
def main() -> None:
    df = pd.read_csv(DATA)
    df["revenue"] = df["quantity"] * df["unit_price"]
    df["month"] = df["order_date"].str.slice(0, 7)
    monthly = df.groupby("month")["revenue"].sum().sort_index()
    best = monthly.idxmax()
    print("Monthly revenue:")
    for month, rev in monthly.items():
        print(f"  {month}  {rev:>12,.2f}")
    print(f"\nBest month: {best}  ({monthly[best]:,.2f})")
    print(f"Total revenue (raw, incl. duplicate orders): {monthly.sum():,.2f}")

if __name__ == "__main__":
    main()
```

☐ **Step 2: Write the expected-output note**

Create `claude-code-workshop/fallbacks/b2-expected-output.md` describing what a correct run shows: a 12-row monthly table July 2024 → June 2025, rising into the holiday season, **best month = 2024-12**, and the verification line "running ≠ correct — confirm December is the peak." Include the exact `2024-12` revenue from `python claude-code-workshop/_build/check_data.py` output (do not invent it — copy from the run).

☐ **Step 3: Write the MCP + PR fallbacks**

Create `claude-code-workshop/fallbacks/b4-mcp-config.json` (a filesystem MCP server config — exact schema verified at build against current `claude mcp add` / `.mcp.json` format), `claude-code-workshop/fallbacks/b5-antigravity-mcp-config.json` (the equivalent for Antigravity), and `claude-code-workshop/fallbacks/b4-pr-example.md` (a model pull-request title + body describing the monthly report, so the room can proceed if live GitHub steps stall).

☐ **Step 4: Write the fallback check**

Create `claude-code-workshop/_build/check_fallbacks.py`:

```
"""Run the Block 2 fallback and assert the December peak.
Run: python claude-code-workshop/_build/check_fallbacks.py"""
from pathlib import Path
import subprocess, sys

ROOT = Path(__file__).resolve().parents[1]
SCRIPT = ROOT / "fallbacks" / "b2-sales-summary.py"

def main() -> int:
    out = subprocess.run([sys.executable, str(SCRIPT)], capture_output=True, text=True)
    if out.returncode != 0:
        print("FAIL: fallback script errored\n", out.stderr)
        return 1
    if "Best month: 2024-12" not in out.stdout:
        print("FAIL: December peak missing from output\n", out.stdout)
        return 1
    print("FALLBACKS OK")
    return 0

if __name__ == "__main__":
    sys.exit(main())
```

☐ **Step 5: Run it**

Run: `python claude-code-workshop/_build/check_fallbacks.py` Expected: `FALLBACKS OK`

☐ **Step 6: Commit**

```
git add claude-code-workshop/fallbacks/ claude-code-workshop/_build/check_fallbacks.py
git commit -m "feat(cc-workshop): add deterministic fallbacks and fallback check"
```

Task 5: Token-consumption notebook (builder + sample data + smoke)

Files:

- Create: `claude-code-workshop/_build/build_token_notebook.py`
- Create: `claude-code-workshop/notebook/results.sample.csv`
- Create: `claude-code-workshop/notebook/token-consumption-lab.ipynb` (generated)
- Create/Test: `claude-code-workshop/_build/smoke_notebook.py`

Interfaces:

- Produces: a notebook whose **presentation layer** reads a results CSV (columns: `setting_group, setting, input_tokens, output_tokens, cached_input_tokens, usd_cost, latency_s`) and renders a table + bar charts; an **API measurement layer** (gated behind an explicit key check) that produces the real `results.csv`.
- The smoke test runs the presentation logic against `results.sample.csv` with NO API key.

☐ **Step 1: Write the illustrative sample results**

Create `claude-code-workshop/notebook/results.sample.csv` with clearly-labelled ILLUSTRATIVE rows covering the four setting groups (`model`: opus/sonnet/haiku; `effort`: low/medium/high/max; `caching`: uncached/cached; `context`: small/stuffed). Use plausible relative magnitudes; mark in the notebook text that these are placeholders until the real run (Task 6). Header exactly: `setting_group,setting,input_tokens,output_tokens,cached_input_tokens,usd`

☐ **Step 2: Write the notebook builder**

Create `claude-code-workshop/_build/build_token_notebook.py` using `nbformat`. The builder emits `notebook/token-consumption-lab.ipynb` with this cell sequence:

1. **Markdown** — title + "what this notebook shows" (token cost changes with model, effort, caching, context) + "you are reading measured numbers; re-running is optional and needs your own API key."
2. **Code (presentation, no key needed)** — load `results.csv` if present else `results.sample.csv` into pandas; display the tidy table.
3. **Code (charts)** — matplotlib bar charts: cost-per-model, cost-per-effort, cached-vs-uncached (read from the dataframe).
4. **Markdown** — "Optional: measure it yourself" explaining the gated cell.
5. **Code (API measurement, gated)** — guarded by `import os; KEY = os.environ.get("ANTHROPIC_API_KEY")` and `if not KEY: print("No key — skipping live measurement; reading committed results.")`. When a key is present: send the SAME short business prompt under each setting using the `anthropic` SDK, read `response.usage` fields, compute `usd_cost` from CURRENT pricing (pull exact model IDs + prices from the `claude-api` skill at build — do NOT hardcode from memory), write `results.csv`.

The builder must be deterministic (no `Date.now()`-style nondeterminism in committed cells).

☐ **Step 3: Build the notebook**

Run: `python claude-code-workshop/_build/build_token_notebook.py` Expected: writes `claude-code-workshop/notebook/token-consumption-lab.ipynb`, prints a confirmation.

☐ **Step 4: Write the smoke test**

Create `claude-code-workshop/_build/smoke_notebook.py` that imports the presentation logic (load CSV → build dataframe → render a figure to a temp PNG via matplotlib Agg) against `results.sample.csv` with `ANTHROPIC_API_KEY` unset, and asserts: dataframe has the four setting groups and a PNG was written. Print `NOTEBOOK SMOKE OK`.

☐ **Step 5: Run the smoke test**

Run: `python claude-code-workshop/_build/smoke_notebook.py` Expected: `NOTEBOOK SMOKE OK`

☐ **Step 6: Commit**

```
git add claude-code-workshop/_build/build_token_notebook.py claude-code-workshop/notebook/
git commit -m "feat(cc-workshop): token-consumption notebook builder, sample data, smoke test"
```

Task 6: Populate the notebook with real measurements (author step)

Files:

- Modify: `claude-code-workshop/notebook/token-consumption-lab.ipynb` (executed outputs)
- Create: `claude-code-workshop/notebook/results.csv` (real measured data)

Interfaces:

- Consumes: `build_token_notebook.py` output; a real `ANTHROPIC_API_KEY`.
- Produces: a committed notebook with real outputs + charts, and `results.csv`. If skipped, the bundle still verifies via `results.sample.csv`.

☐ **Step 1: Confirm current pricing/model IDs**

Use the `claude-api` skill to record the current model IDs and per-token input/output (and cache) prices into the builder's pricing map. Re-run `python claude-code-workshop/_build/build_token_notebook.py`.

☐ **Step 2: Run the live measurement (requires key + small spend)**

```
export ANTHROPIC_API_KEY=...   # PowerShell: $env:ANTHROPIC_API_KEY="..."
jupyter nbconvert --to notebook --execute --inplace \
  claude-code-workshop/notebook/token-consumption-lab.ipynb
```

Expected: the gated cell runs, writes `results.csv`, charts render with real numbers. Cost: a few cents (document the exact figure in instructor notes).

☐ **Step 3: Verify the smoke test still passes against the real CSV**

Run: `python claude-code-workshop/_build/smoke_notebook.py` Expected: `NOTEBOOK SMOKE OK`

☐ **Step 4: Commit**

```
git add claude-code-workshop/notebook/token-consumption-lab.ipynb claude-code-workshop/notebook/resul
git commit -m "feat(cc-workshop): populate token notebook with real measurements"
```

Task 7: Slide deck (Quarto reveal.js) + render check

Files:

- Create: `claude-code-workshop/slides/claude-code-workshop.qmd`

Interfaces:

- Consumes: the canonical prompts from `exercises/b*.md` (Tasks 2–3) and the notebook charts (Task 5/6).
- Produces: a reveal.js deck rendering to `slides/claude-code-workshop.html`.

☐ **Step 1: Confirm Quarto is available**

Run: `quarto --version` Expected: a version string. If absent, document the install in `README.md` and skip render verification with a noted TODO for the author's machine.

☐ **Step 2: Write the deck**

Create `claude-code-workshop/slides/claude-code-workshop.qmd`. YAML front matter:

```
---
title: "Claude Code, Hands-On"
subtitle: "From a sales file to a self-running tool"
format:
```

```
  revealjs:
    theme: simple
    slide-number: true
    incremental: false
---
```

Then six section groups mirroring the spec's blocks. Rules:

- Each block opens with a section divider slide (`# Block N — Title`).
- Alternate **concept slides** (≤6 bullets, business-framed, plain English) with **"□ Your turn"** slides that show the EXACT prompt to type (reuse the card prompts verbatim).
- Put facilitator script in speaker notes (`::: notes ... :::`).
- Do NOT print durations or clock times on slides (instructor discretion — Global Constraints).
- Verify every Claude Code command shown (efforts, `/fast`, `/loop`, `claude mcp add`, `/cost`, `/context`, `/compact`) against the running CLI / docs before writing it.

Required slides per block (minimum):

- **B1 Doorway & Setup:** what Claude Code is; "the terminal is a conversation"; □ `claude --version` / `claude` / "What files are in this folder…".
- **B2 The Loop & The Goal:** the loop diagram (context → act → verify → repeat); goals & plan mode; permissions/approval; □ the monthly-report goal prompt; the verification prompt (best month = 2024-12).
- **B3a Turning the Dials:** reasoning efforts; models (Opus/Sonnet/Haiku); `/fast`; □ re-run "think harder".
- **B3b Reading the Meter:** embed the notebook charts (cost-per-model, cost-per-effort, cached-vs-uncached); `/usage` `/context` `/compact`; □ run `/usage`.
- **B4 Giving Claude Hands:** what MCP is ("new senses and hands"); □ `claude mcp add …`; □ "create a repo / open a PR"; note the PR is opened without typing git.
- **B5 A Second Cockpit:** Antigravity intro; MCP transfers; □ connect MCP + "add a bar chart"; quota caveat.
- **B6 Make It Run Itself + Wrap:** `/loop` / scheduled work; □ "run every Monday"; verify-everything refrain; recap CSV → analysed → shipped → automated; where to go next.

□ **Step 3: Render to verify**

Run: `quarto render claude-code-workshop/slides/claude-code-workshop.qmd --to revealjs` Expected: exit 0; `claude-code-workshop/slides/claude-code-workshop.html` produced.

□ **Step 4: Commit**

```
git add claude-code-workshop/slides/claude-code-workshop.qmd
git commit -m "feat(cc-workshop): Quarto reveal.js slide deck"
```

(Do not commit the generated `.html` unless the author wants it tracked; add to `.gitignore` if not.)

Task 8: README + pre-work + instructor notes + bundle verifier

Files:

- Create: `claude-code-workshop/README.md`
- Create: `claude-code-workshop/pre-work.md`
- Create: `claude-code-workshop/instructor-notes.md`
- Create/Test: `claude-code-workshop/_build/verify_bundle.py`

Interfaces:

- Produces: the prose docs and a single verifier that runs all checks and asserts every expected file exists with required headings.

□ **Step 1: Write `README.md`**

Facilitator overview: audience (non-technical professionals), the one-project spine, the block sequence (NO fixed times — point to instructor notes), prerequisites (paid Claude access, Node), how to render the deck (`quarto render …`),

how to open the notebook, the per-participant cost-budget pointer, and a one-line "this is a standalone professional workshop, outside the IMB2026 free-tools student track."

☐ **Step 2: Write `pre-work.md`**

One page: create the Claude account, install Node, confirm a terminal opens, `claude --version`. Reassure: "if any step fails, we fix it together in the first block."

☐ **Step 3: Write `instructor-notes.md`**

Include: the SUGGESTED durations template (instructor discretion; the only place timings live) with a 90-minute lunch creating the "turn the dials / read the meter" seam; Block 1 setup troubleshooting (Node/npm, auth, Windows vs Mac, pair/watch fallback); the Antigravity weekly-quota caveat + re-verify-1-week-before reminder; the realistic per-participant token cost budget for the day + the notebook's build cost; fallback cues (when to reach for each `fallbacks/` artifact); the verify-everything refrain (running ≠ correct; confirm December is the peak).

☐ **Step 4: Write the bundle verifier**

Create `claude-code-workshop/_build/verify_bundle.py` that:

- asserts these files exist: `data/orders.csv`, `slides/claude-code-workshop.qmd`, `notebook/token-consumption-lab.ipynb`, `notebook/results.sample.csv`, all six `exercises/b*.md`, the five `fallbacks/*`, `README.md`, `pre-work.md`, `instructor-notes.md`;
- asserts `README.md`/`instructor-notes.md`/`pre-work.md` contain expected H1/H2 anchors;
- shells out to `check_data.py`, `check_cards.py`, `check_fallbacks.py`, `smoke_notebook.py` and fails if any fail;
- prints `BUNDLE OK`.

☐ **Step 5: Run it**

Run: `python claude-code-workshop/_build/verify_bundle.py` Expected: `BUNDLE OK`

☐ **Step 6: Commit**

```
git add claude-code-workshop/README.md claude-code-workshop/pre-work.md claude-code-workshop/instruct
git commit -m "feat(cc-workshop): README, pre-work, instructor notes, bundle verifier"
```

Task 9: Accuracy & freshness pass

Files:

- Modify: `claude-code-workshop/slides/claude-code-workshop.qmd`, `claude-code-workshop/_build/build_tok` `claude-code-workshop/exercises/*.md`, `claude-code-workshop/instructor-notes.md` (only where facts need correcting)

Interfaces:

- Consumes: the `claude-api` skill; the running `claude` CLI; `https://docs.claude.com/`.

☐ **Step 1: Verify Claude facts**

Cross-check, and correct in place: current model IDs + pricing (notebook builder), effort tiers + `/fast`, `/loop`/scheduling syntax, `claude mcp add` + `.mcp.json` schema (Task 4 configs + b4 card), `/cost` `/context` `/compact` behaviour. Record the verification date in `instructor-notes.md`.

☐ **Step 2: Antigravity reminder**

Confirm `instructor-notes.md` carries the "re-verify Antigravity free-tier limits ~1 week before delivery" note with today's known state (weekly quota; no committed permanent free tier).

☐ **Step 3: Re-run the full verifier**

Run: `python claude-code-workshop/_build/verify_bundle.py` Expected: `BUNDLE OK`

☐ **Step 4: Commit**

```
git add claude-code-workshop/
git commit -m "chore(cc-workshop): accuracy and freshness pass"
```

Self-Review

Spec coverage — every spec section maps to a task:

- §2 audience / Terminal CLI / guided setup → Task 2 (b1 card), Task 8 (instructor troubleshooting)
- §3 objectives (loop, goal, efforts, tokens, MCP, GitHub, Antigravity, /loop, verify) → Tasks 3 (cards b2–b6), 5–6 (token notebook), 7 (deck slides per block)
- §4 spine + Adriatica data + fallbacks → Tasks 1, 4
- §5 block sequence → Task 7 deck sections + Task 8 durations template
- §6.1 deck → Task 7; §6.2 notebook → Tasks 5–6; §6.3 cards → Tasks 2–3; §6.4 fallbacks → Task 4; §6.5 instructor notes → Task 8; §6.6 pre-work → Task 8
- §7 accuracy/latest-versions → Task 6 (pricing), Task 9 (facts); §8 risks → fallbacks (Task 4), smoke/verify (Tasks 5,8), quota note (Task 8)

Placeholder scan — sample-vs-real notebook data is intentional (Task 5 sample, Task 6 real), not a placeholder; the `results.sample.csv` magnitudes and the b2 December revenue are explicitly "copy from the real run, do not invent." No "TBD/handle edge cases" steps.

Type consistency — `monthly_revenue(path) -> Series` defined in Task 1 and reused conceptually in Task 4; results CSV header `setting_group,setting,input_tokens,output_tokens,cached_input_tokens,usd_cost,latency_s` is identical across Tasks 5, 6, 8; card schema headings identical in Global Constraints, Task 2 linter, and Task 3.